

Program 1

8-Puzzle Problem Using Best First Search

Solve the 8-puzzle using Greedy Best First Search. Use the heuristic $h(n)$ = number of misplaced tiles. Generate moves and print the sequence of states from the initial configuration to the goal.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_07 > Q1.py > ...

```

1  from queue import PriorityQueue
2
3  goal = (1, 2, 3,
4         4, 5, 6,
5         7, 8, 0)
6
7  initial = (1, 2, 3,
8            4, 0, 6,
9            7, 5, 8)
10
11 moves = {
12     "Up": -3,
13     "Down": 3,
14     "Left": -1,
15     "Right": 1
16 }
17
18 def heuristic(state):
19     count = 0
20     for i in range(9):
21         if state[i] != 0 and state[i] != goal[i]:
22             count += 1
23     return count
24
25 def get_neighbors(state):
26     neighbors = []
27     zero = state.index(0)
28
29     row = zero // 3
30     col = zero % 3
31
32     possible = []
33
34     if row > 0:
35         possible.append(("Up", zero - 3))
36     if row < 2:
37         possible.append(("Down", zero + 3))
38     if col > 0:
39         possible.append(("Left", zero - 1))
40     if col < 2:
41         possible.append(("Right", zero + 1))
42
43     for move, pos in possible:
44         new_state = list(state)
45         new_state[zero], new_state[pos] = new_state[pos], new_state[zero]
46         neighbors.append((move, tuple(new_state)))
47
48     return neighbors
49
50 pq = PriorityQueue()
51 pq.put((heuristic(initial), initial, []))
52
53 visited = set()
54
55 while not pq.empty():
56     h, current, path = pq.get()
57
58     if current in visited:
59         continue
60
61     visited.add(current)
62
63     if current == goal:
64         print("Initial:", initial, "h =", heuristic(initial))
65         for i, (move, state) in enumerate(path, start=1):
66             print(f"Move {i}: {move} -> {state} h={heuristic(state)}")
67         print("\nGOAL REACHED")
68         break
69
70     for move, neighbor in get_neighbors(current):
71         if neighbor not in visited:
72             new_path = path + [(move, neighbor)]
73             pq.put((heuristic(neighbor), neighbor, new_path))

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

● gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur
.py"
Initial: (1, 2, 3, 4, 0, 6, 7, 5, 8) h = 2
Move 1: Down -> (1, 2, 3, 4, 5, 6, 7, 0, 8) h=1
Move 2: Right -> (1, 2, 3, 4, 5, 6, 7, 8, 0) h=0

```

GOAL REACHED

```

❖ gauravmathur@Annus-Mac Artificial_Intelligence_Lab %

```

A* Algorithm on a Weighted Graph

Implement A* using $f(n) = g(n) + h(n)$ on the same weighted graph from Lab 6. Find the OPTIMAL path from S to G. Print the explored order, optimal path, and total cost.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_07 > Q2.py > astar

```

1  import heapq
2  graph = {
3      'S': [('A', 3), ('B', 5)],
4      'A': [('C', 4), ('D', 2)],
5      'B': [('D', 6), ('G', 9)],
6      'C': [('G', 7)],
7      'D': [('E', 1)],
8      'E': [('G', 3)],
9      'G': []
10 }
11 heuristic = {
12     'S': 10,
13     'A': 7,
14     'B': 6,
15     'C': 5,
16     'D': 4,
17     'E': 2,
18     'G': 0
19 }
20 def astar(start, goal):
21     pq = []
22     heapq.heappush(pq, (heuristic[start], 0, start, []))
23
24     visited = set()
25
26     while pq:
27         f, g, node, path = heapq.heappop(pq)
28
29         if node in visited:
30             continue
31
32         visited.add(node)
33
34         path = path + [node]
35
36         print("Visited:", node)
37
38         if node == goal:
39             print("\nOptimal Path:", " -> ".join(path))
40             print("Total Cost:", g)
41             return
42
43         for neighbor, cost in graph[node]:
44             if neighbor not in visited:
45                 new_g = g + cost
46                 new_f = new_g + heuristic[neighbor]
47
48                 heapq.heappush(
49                     pq,
50                     (new_f, new_g, neighbor, path)
51                 )
52
53     astar('S', 'G')

```

```

● gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/
Visited: S
Visited: A
Visited: D
Visited: E
Visited: G

Optimal Path: S -> A -> D -> E -> G
Total Cost: 9
● gauravmathur@Annus-Mac Artificial_Intelligence_Lab %

```

Implement AO* on the given AND-OR graph. Find the best solution sub-graph by repeatedly choosing the cheapest connector and revising heuristic estimates as nodes are expanded.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_07 > Q3.py > ao_star

```

1 graph = {
2     'A': [
3         ('OR', ['B'], 1),
4         ('AND', ['C', 'D'], 1)
5     ],
6
7     'B': [
8         ('AND', ['E', 'F'], 1)
9     ],
10
11     'C': [],
12     'D': [],
13     'E': [],
14     'F': []
15 }
16 heuristic = {
17     'A': 0,
18     'B': 5,
19     'C': 3,
20     'D': 4,
21     'E': 7,
22     'F': 9
23 }
24 def ao_star(node):
25     if graph[node] == []:
26         return 0
27
28     costs = []
29
30     for connector_type, children, edge_cost in graph[node]:
31
32         total = 0
33
34         for child in children:
35             total += edge_cost + ao_star(child)
36
37         costs.append((total, connector_type, children))
38
39     best = min(costs, key=lambda x: x[0])
40
41     return best[0]
42
43 print("Total Minimum Cost from A:", ao_star('A'))

```

Output:—

```

gaurvmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmatl
Total Minimum Cost from A: 2
gaurvmathur@Annus-Mac Artificial_Intelligence_Lab %

```

Run All 5 Algorithms - See Where Each Works and Fails

Run DFS, BFS, Best First and A* on the same weighted graph (Program 2). Then run AO* on the AND-OR graph (Program 3). Compare paths, costs, and discover which properties (completeness, optimality) each satisfies in practice.

```

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_07 > Q4.py > bfs
1 from collections import deque
2 import heapq
3 graph = {
4     'S': [('A', 3), ('B', 5)],
5     'A': [('C', 4), ('D', 2)],
6     'B': [('D', 6), ('G', 9)],
7     'C': [('G', 7)],
8     'D': [('E', 1)],
9     'E': [('G', 3)],
10    'G': []
11 }
12 heuristic = {
13     'S': 10,
14     'A': 7,
15     'B': 6,
16     'C': 5,
17     'D': 4,
18     'E': 2,
19     'G': 0
20 }
21 # DFS
22 def dfs(start, goal):
23     stack = [(start, [start], 0)]
24     visited = set()
25
26     while stack:
27         node, path, cost = stack.pop()
28
29         if node == goal:
30             return path, cost
31
32         if node not in visited:
33             visited.add(node)
34
35             for neighbor, w in reversed(graph[node]):
36                 stack.append((neighbor, path + [neighbor], cost + w))
37 # BFS
38 def bfs(start, goal):
39     q = deque([(start, [start], 0)])
40     visited = set()
41
42     while q:
43         node, path, cost = q.popleft()
44
45         if node == goal:
46             return path, cost
47
48         if node not in visited:
49             visited.add(node)
50
51             for neighbor, w in graph[node]:
52                 q.append((neighbor, path + [neighbor], cost + w))
53
54 # Best First Search
55 def best_first(start, goal):
56     pq = []
57     heapq.heappush(pq, (heuristic[start], start, [start], 0))
58
59     visited = set()
60
61     while pq:
62         h, node, path, cost = heapq.heappop(pq)
63
64         if node == goal:
65             return path, cost
66
67         if node not in visited:
68             visited.add(node)
69
70             for neighbor, w in graph[node]:
71                 heapq.heappush(
72                     pq,
73                     (heuristic[neighbor],
74                      neighbor,
75                      path + [neighbor],
76                      cost + w)
77                 )
78
79 # A*
80 def astar(start, goal):
81     pq = []
82     heapq.heappush(pq, (heuristic[start], 0, start, [start]))
83
84     visited = set()
85
86     while pq:
87         f, g, node, path = heapq.heappop(pq)
88
89         if node == goal:
90             return path, g
91
92         if node not in visited:
93             visited.add(node)
94
95             for neighbor, w in graph[node]:
96                 new_g = g + w
97                 new_f = new_g + heuristic[neighbor]
98
99                 heapq.heappush(
100                     pq,
101                     (new_f,
102                      new_g,
103                      neighbor,
104                      path + [neighbor])
105                 )
106
107 print("DFS:", dfs('S', 'G'))
108 print("BFS:", bfs('S', 'G'))
109 print("Best First:", best_first('S', 'G'))
110 print("A*:", astar('S', 'G'))

```

```

● gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/Desktop/Art
DFS: (['S', 'A', 'C', 'G'], 14)
BFS: (['S', 'B', 'G'], 14)
Best First: (['S', 'B', 'G'], 14)
A*: (['S', 'A', 'D', 'E', 'G'], 9)
❖ gauravmathur@Annus-Mac Artificial_Intelligence_Lab %

```